MS2 Summary — Padel Predictor

Noah Rodriguez · github.com/Noah-Rod/padel-predictor

1 What I achieved in this milestone

- **A scheduled, backfillable feature pipeline.** `.github/workflows/feature-pipeline.yml` runs daily at 04:15 UTC and takes a date range for a backfill (`padel backfill`, 30-day chunks, resumable). Each run ingests seven days back and seven days of fixtures, de-duplicates on `match_id` (newest record wins, so a fixture later becomes one labelled row) and rebuilds every feature from scratch: Elo and form depend on the order of matches, so appending would quietly desynchronise all rows after a late result.
- **Hopsworks feature store and W&B registry, as in the proposal.** With `PADEL_FEATURE_STORE_URI=hopsworks://<project>`, `raw_matches`, `raw_rankings`, `match_features` and `predictions` are feature groups with a primary key and an event time, written by upsert, so a re-ingested match replaces its row (`src/padel_predictor/common/storage.py`). With `PADEL_TRACKING=wandb`, every training run is a W&B run and every candidate a `model` artifact; the promoted one carries the `champion` alias that inference loads (`training/registry.py`). My first version had used Parquet and a local MLflow registry instead. I moved back to the proposal's stack because a GitHub Actions runner starts with an empty disk, so the local stores lost all features and models after every run. The Parquet store and an on-disk registry with the same contract remain as offline backends, so the tests and a fresh clone run without any account.
- **One feature definition, now leakage-tested for rankings too.** `common/features.py` is imported by training and inference. New in MS2: rankings joined on their publication time (E1), the provider's Elo stored as a baseline next to every row (A2), and the pair aggregation written down (B1).
- **Reproducible.** Docker image, `uv.lock`, 105 unit tests plus lint on Python 3.11 and 3.12 in CI; without an API key the `sample` source runs the whole system offline.

Look at first: `attach_rankings` in `src/padel_predictor/common/features.py` and the ranking leakage tests at the end of `tests/test_features.py`; then run `padel data-report` (`src/padel_predictor/features/report.py`).

What this milestone did not reach. The scheduled runs still use the synthetic `sample` source and the offline backends, so nothing accumulates between runs yet. The live adapter (`HttpMatchSource` in `features/sources.py`) implements the whole contract (matches, ranking history, free and paid plan), but its payload mapping is not yet verified against padelapi.org, and the padelapi.org, Hopsworks and W&B keys are not yet set as repository secrets. The Hopsworks and W&B backends are covered by unit tests against stand-ins and a W&B offline run, not yet by a run against the hosted services. That is my first task after MS2, and it is why the real counts and the real baseline number are still missing below (A2, E2). Two proposal claims the code did not honour yet are now fixed: sides A/B are assigned by a hash of the pair names (`orient_teams` in `common/schemas.py`), and the test split is cut at a tournament boundary (`training/dataset.py`).

2 Response to MS1 reviews

Labels as in the editorial summary. Reviewer A's suggestion is A2, reviewer B's suggestion is B1. Code: commits `a2fb2a5` and `9a3913f`; documentation: `0b5feb8`.

- **A2 · Baseline is not measured on my test set — Partly** (also reviewer A's suggestion). The provider's Elo now comes from the ranking-history snapshot published strictly before each match. It is stored as `provider_elo_prob` beside each feature row, never as a model input, and scored on exactly the model's test rows. A test match without a snapshot gets 0.5 instead of being dropped, so the bar is not set on a friendlier subset, and the coverage is reported. Training logs it as `baseline_provider_elo_log_loss`; `padel data-report` prints it. *Not yet:* the number on real data (live source, see section 1), and registering the provider's Elo as the first champion (MS3). *Where:* `training/dataset.py` (`provider_elo_probabilities`), `features/report.py`.
- **E1 · Weekly snapshots can leak same-week results — Changed.** Rankings are now their own raw dataset with a `published_at` column. Each player gets the latest snapshot published *strictly before* the scheduled start (`merge_asof` without exact matches), never the one matching the label date. Backfilled snapshots without a publication time count as published seven days after their label date, so a snapshot labelled with a tournament week is never visible inside that week; re-reading a snapshot keeps the first time it was seen. Tests: a snapshot published after the week, one published exactly at the start, and rewriting every later snapshot leaves the match's row unchanged. *Where:* `common/features.py` (`attach_rankings`), `common/schemas.py`, `tests/test_features.py`.
- **A1 · No minimum margin on ~2,000 test matches — Changed.** A paired bootstrap (2,000 resamples) of the per-match log-loss difference, model minus provider Elo. The criterion is met only if the whole 95 % interval lies below zero; it is logged as `vs_provider_elo_ci_low/high` and the tag `beats_provider_elo`. I chose an interval

over a fixed margin, because a margin would be a guess about a test-set size I do not know yet. A test checks that a 0.001 edge on 2,000 matches does not count as a win. *Where:* `training/dataset.py` (`bootstrap_log_loss_difference`), `config/settings.yaml`.

- **E2 · Paid data tier: real volume and a fallback — Partly.** `padel data-report` prints labelled and upcoming matches per season and tier, and the number of ranking snapshots, read from the store rather than estimated. Fallback: `PADEL_API_PLAN=free` clamps the backfill to 183 days and stores only the current ranking, published at fetch time, so no historical match gets points it could not have had. Without any key, the `sample` source runs everything. *Not yet:* the real counts, for the same reason as A2. *Where:* `features/report.py`, `features/pipeline.py`, README "Going live".
- **B1 · How player values become team features — Changed.** Elo: mean of the two players. Ranking points: sum, from the snapshot. Form: mean of each player's win rate over their last 5 matches with any partner, 0.5 without history. Rest: the smaller of the two players' gaps, capped at 60 days, 14 if never seen. Partnership: matches this exact pair has played together. A new partnership starts at 0 but keeps each player's Elo, form and rest; a test covers this. *Where:* README "What the model sees", `common/features.py` docstring, `tests/test_features.py`.
- **E3 · Report what partnership and rest add — Not yet.** Ablations belong to the training pipeline (MS3). The feature set stays at six, and the provider's Elo was taken out of the features so it stays a clean bar. In MS3, partnership, form and rest are each dropped in turn and every run's interval against the provider's Elo is compared in W&B. *Where:* planned in `training/pipeline.py`.
- **E4 · Log which model served which day — Changed.** Every stored prediction row carries `model_name`, `model_version`, `model_run_id` and `predicted_at` in the `predictions` feature group (primary key `match_id`, `run_date`), so the last pre-match prediction per match is the latest `predicted_at` before `played_at`. Scoring it live is MS4 monitoring. *Where:* `inference/predict.py` (`predict_frame`), `tests/test_inference.py`.